

Unsupervised Neural Network for Multi-Genre Music Generation

Syed Azmain Ishmam

Student ID : 22201654

CSE426 Section: 6 Serial no.: 615

Department: CSE (BRAC University)

syed.azmain.ishmam@g.bracu.ac.bd

Mahmud Al Fuad Rubai

Student ID : 22201329

CSE425 Section: 5 Serial no.: 510

Department: CSE (BRAC University)

mahmud.alfuad.rubai@g.bracu.ac.bd

Abstract—This paper is an unsupervised Multi-Genre music generative framework containing four stages. It is done using MIDI data from the MAESTRO v3.0.0 dataset. Instead of putting focus on the benchmark scores this paper puts priority into the deep analysis of the behavior of each model, its failures and the structural quality it outputs. Firstly, it begins with and LSTM autoencoder for reconstructing basic music which is then extended to a Variational Autoencoder (VAE) for creating diversity for the latent space which again extends to the development of a decoder-only transformer in order to generate longer ones. Then finally RLHF(Reinforcement Learning from Human Feedback) is applied in order to generate outputs that align with the preference of human beings. Each of these stages are examined rigorously on how it works, why it works, where it doesn't work and what the outputs reveal about what the model is understanding. For evaluation automated metrics such as pitch histogram, similarity, diversity of rhythm and ratio of repetition was used with a blind human listening survey. The compositions that were generated is publicly available at this link: <https://tinyurl.com/genMidis>

Index Terms—MAESTRO, MIDI, RLHF, LSTM, Autoencoder, Transformer, Generative, Token, MSE

I. INTRODUCTION

One of the most structurally and culturally enriched domain of human expression is music. At a moment in a musical piece a lot of things such as note pitches, melody, harmonious progression, rhythm and genre-specific conventions operate simultaneously. Generating music like which is coherent to all of these things is one of the hardest things in generative deep learning.

The main thing in machine learning is that it requires data that is labeled. But in music a single piece does not belong to a single genre. In this case unsupervised learning comes to play. As a result the model learns musical structure from the raw data and discovers the patterns by itself without knowing what it is looking for.

This project is built on the MAESTRO classical piano dataset and contains four progressively generative models that moves from just simple reconstruction to generating music that human prefer more.

- **Task 1 (LSTM Autoencoder):** This model learns to compress and reconstruct the short piano-roll sequences

and then establisher if the model is able to capture note patterns that are basic or not in a compact latent code.

- **Task 2 (Variational Autoencoder[VAE]):** It puts a probability. As a result the reconstructed sequence is more diverse.
- **Task 3 (Transformer):** This model captures the full sequence distribution in an auto regressive way. It tokenizes over the MIDI events and captures long range dependencies which is missed by the models that are LSTM-based.
- **Task 4 (RLHF Fine Tuning):** It uses the feedback of human to further refine the output that was given by the transformer model. This makes the output suit better to what human being prefer.

The main question of the paper is not which model works better, but which model produces what. Even though the scores are given the main thing is to analyze the outputs data.

All of these models are compared with two baseline models which are 1)Random note generator and 2) Markov chain. Comparing these two helps to evaluate what the models learn newly.

II. METHODOLOGY

A. Dataset and Preprocessing

- **MAESTRO v3.0.0**

All the models use the MAESTRO v3.0.0 dataset. It contains a total of 1,276 MIDI recordings of classical pianos notes. The total duration is approximately 200 hours. The piano used for the recordings is the Yamaha Disklavier grand pianos with a timing accuracy of approximately 3ms. Only the MIDI package of 57MB was used as we did not require any audio for the required tasks.

- **Piano-Roll Representation (Task 1 and 2)**

The MIDI files are converted to binary piano-roll matrices using the pretty_midi library. Their FPS is 16. A two dimensional matrix of shape (t,88) is produced by them. Here, T is the total time steps and 88 is the number of piano keys. Each of the cells are made 0 and 1 for silent and active respectively.

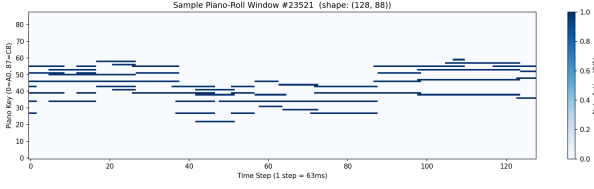


Fig. 1. Binary piano-roll window

Each of the piano rolls are segmented into 128 steps of non-overlapping windows. They cover upto 8 seconds of at 16 fps. The windows that have less than 2% of active cells are discarded as they contain no notes almost. So, they are not useful for gradient signals. This filtering is crucial. IF it is not done then the portions that are silent makes the loss function to converge to a near-zero prediction without learning any note.

The final dataset consists totally of 62689 training windows, 7876 validation windows and 7792 test windows with a shape of (128, 88). One of the key properties of this dataset is that nearly 97-98% of the cells are at zero at any time stamp. This create a severe class imbalance which is needed to be explicitly addressed in the loss function.

• Token Sequence Representation (Tasks 3 and 4)

The miditok library’s REMI scheme was used to tokenize the MIDI files for the Transformer task. REMI encodes musics as a flat integer sequence. There each token represents one of a Bar marker, a position offset within the bar, a value of Pitch, a velocity bin or a value of duration. Both pitch content and temporal structure is captured in a format which is naturally suited for the next-token prediction.

The sequences are segmented in an overlapping windows of 256 tokens. They have 50% stride. The final resulting vocabulary contains 284 unique tokens. REMI representation avoids entirely the sparsity problem of the piano rolls. In this all tokens are meaningful. As a result, it is a more appropriate representation for the sequence modeling.

B. Baseline Models

• Random Note Generator

Notes are derived by the predefined duration set 0.25, 0.5, 1.0 seconds and these are sampled consistently from the 88-key piano range. Later, these notes are arranged at periodic onsets. The baseline shows the absolute performance score having zero musical knowledge. For interpreting each model’s progress as the amount of musical pattern that has been learned over random chances, it tries to establish the look “No structure at all” on the basis of metric scores.

• First-Order Markov Chain

Initially, the pitch transition matrix is calculated from training data. It is rendered by counting how often pitch i is followed by pitch j ; then each row is normalized into a probability distribution. The training set’s speculative distribution is used for determining note durations. Local Melodic step tendency is captured by the baseline. It knows that a distant pitch might not follow C but D or E may do that. It has lacking of harmonic awareness memory beyond the note which comes before it, the tendency to build up phrases as well as rhythm beyond single note durations. Its predicted position from the findings whether our model got to know anything over simple pitch transition statistics or not.

C. Task 1(LSTM Autoencoder)

• Architecture

The encoder combines two stacked LSTM layers of hidden size 256 for operating a piano-roll window X of form (128, 88). The hidden state of the last layer is projected to a hidden vector z of size 64 at the last time step:

$$z = f_{\phi}(X) = \text{Linear}(h_T^{\{last\}}) \quad z \in \mathbb{R}^{64}$$

In order to make sure that the hidden code is visible for the entire operating process. The decoder retrieves z then repeats it for 128 time steps in a whole. This reformation is obtained by latent dimensional two layer LSTM which has the dimension of 256. A linear projection of 88 raw logits also assists to construct it:

$$\hat{X} = g_{\theta}(z) \quad \hat{X} \in \mathbb{R}^{128 \times 88}$$

Raw logits are straight away sent to the loss function, This implements sigmoid internally for numerical stability.

• Loss Function - Why Not MSE?

Reconstruction loss refers to the square error in between reconstruction and input:

$$L_{AE} = \sum_{t=1}^T \|x_t - \hat{x}_t\|^2$$

Practically, when MSE is directly used on binary piano, it becomes a complete failure. Where 97% of cells are zero, if still zero is predicted everywhere, the MSE becomes extremely low regardless of activating a single note. The cause of this is that the gradient from 3% of active cells are surpassed by the gradient from the silent majority. As a result the model converges toward zero producing almost zero for each cell.

MSE is replaced by Focal loss. This is mainly made for uneven object detection. Gradient Contribution from basic instances is covered up by the modulating factor. This factor reaches zero for a quiet cell which was surely

determined to be nearly zero. Thus, it adds very little to the gradient.

An active note cell was mistakenly projected near at zero. That’s why the factor gets close to it, providing it full gradient weight. This forces the model to learn note activations instead of collapsing to silence when combined with positive class weighting $\alpha = 10$.

- **Training Configuration**

To avoid expanding gradients, the model is trained over 80 epochs using batch size 64, Adam optimizer at learning rate 1×10^{-3} , and gradient clipping at maximum norm 1.0. If validation loss does not improve for ten consecutive epochs, a ReduceLROnPlateau scheduler cuts the learning rate in half. After every LSTM layer, a dropout of 0.3 is applied. For generation, the optimal checkpoint based on validation loss is stored.

- **Music Generation**

For the generation of new music, it is not used at all. A sample of a random latent code $z \sim \mathcal{N}(0, I)$ is sampled first. Then it was decoded. The outputs are then converted to probabilities using sigmoid. After that they are thresholded at 0.3 instead of 0.5. This lower threshold is required as the model tends to underestimate the probabilities of notes when trained on imbalanced data (even after training with Focal Loss) otherwise most of the notes generated would be the ones with low probabilities.

D. Task 2(Variational Autoencoder VAE)

- **Architecture and Latent Distribution**

To make the VAE predictions two vectors of the same dimension, namely a mean vector $\mu(X)$ and a log-variance vector $\log \sigma^2(X)$ are both generated by linear layers applied to the final hidden state of the LSTM network. These are the parameters which index the posterior distribution of latent codes:

$$q_\phi(z | X) = \mathcal{N}(\mu(X), \sigma^2(X)) \quad (1)$$

The reparameterization trick is used for the sampling operation, which is broken down into a fixed distribution:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad \sigma = \exp(0.5 \cdot \log \sigma^2) \quad (2)$$

The architecture of the decoder is the same as Task 1. The main difference is that in generation, z is sampled from $\mathcal{N}(0, I)$ leading to meaningful musical output due to regularization in the latent space, while in Task 1 random latent codes may lead to incoherent outputs due to lack of the structure in the latent space.

- **VAE Loss and KL Annealing**

The objective of VAE training is to strike a tradeoff between the quality of the reconstruction and the regularity of the latent space:

$$L_{VAE} = L_{recon} + \beta \cdot D_{KL}(q_\phi(z | X) || p(z)) \quad (3)$$

The KL divergence has a closed-form solution for Gaussian distributions:

$$D_{KL} = -0.5 \cdot \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2) \quad (4)$$

One of VAE’s major pitfalls is posterior collapse—when the decoder is sufficiently expressive that it can generate sequences without referencing z . After this, the encoder cannot work out a useful gradient, and it turns into $\mu = 0, \log \sigma^2 = 0$, producing a posterior the same as the prior, no matter what its input is. The model still reconstructs but is no longer able to produce: taking samples of z from $\mathcal{N}(0, I)$ yields the same output each time.

This is prevented with KL annealing: β begins at 0 (as a pure autoencoder), followed by a linear increase for the first 30 warmup epochs. By doing this, the encoder will learn a meaningful representation based solely on the reconstruction error before KL regularization starts to regularize the latent space.

- **Multi-Genre Generation and Latent Interpolation**

Samples from the latent space are drawn from 8 different regions to generate 8 samples $z \sim \mathcal{N}(0, I)$. Since the only type of piano music in MAESTRO is classical music the term ‘multi-genre’ is used to refer to the diversity in the compositional styles of the music (Baroque counterpoint vs. Romantic melody vs. contemporary classical music) the VAE’s regularized latent space represents as distinct regions.

The latent interpolation experiment consists on encoding two real training pieces to generate mean vectors μ_A and μ_B , and decodes 8 intermediate latent codes:

$$z_\alpha = (1 - \alpha) \cdot \mu_A + \alpha \cdot \mu_B \quad \text{for } \alpha \in \left\{0, \frac{1}{7}, \frac{2}{7}, \dots, 1\right\} \quad (5)$$

This experiment specifically checks the quality of the latent space. Good structure if the space is smooth musically, i.e. by interpolation. Transitions that are abrupt or incoherent and intermediate outputs that lack an adequate structure indicate the space is not regularized adequately.

E. Task 3(Transformer)

- **Architecture**

Here we implemented a decoder only architecture. It consists of a token embedding layer which converts token indexes to vector of size $d_{\text{model}} = 256$, sinusoidal positional encoding added to each token, four layer of transformer decoders with eight attention heads per layer and a hidden dimension of 512 in each feed forward network, and finally, a linear projection to the logits space.

An upper triangular casual attention mask is used at every layer of the self-attention layers such that each position t can only attend up to positions 0 through t . This limitation is not only important in making sure the model trains

correctly but is necessary for generation. Without this masking, the model will be attending to future tokens during training, learns to cheat through near zero training loss, and output gibberish at generation time since there will be no future tokens.

- **Perplexity Evaluation**

The primary metric used to evaluate Task-3 is perplexity on the held-out validation data sequences.

$$\text{Perplexity} = \exp\left(\frac{1}{T} \cdot L_{TR}\right) \quad (6)$$

A lower perplexity score means that the average surprise value the model receives when presented with new sequences is smaller, meaning that the model’s uncertainty is about the same as picking one out of k choices uniformly at random. A lower perplexity score thus corresponds to a more accurately estimated generative distribution by the model. However, it is not always the case that a higher perplexity score will correspond to lower quality output from a human point of view.

$$p(X) = \prod_{t=1}^T p(x_t | x_{<t}) \quad (7)$$

Training minimizes the negative log-likelihood overall positions simultaneously,

$$L_{TR} = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}) \quad (8)$$

This model is entirely different from Tasks 1 and 2 since rather than encoding a window using latent code, it models the conditional probability of each event conditioned on the whole sequence of previous events, thus modeling the generation process itself. The generation utilizes top- k sampling where $k = 40$ and the temperature equals 0.9. This sampling technique restricts the sampling space to the k highest probabilities to avoid predicting too low-probability tokens, which might break the causal chain typical for natural music generation process. Perplexity is used as an indicator of learning a correct generative distribution, but it does not necessarily coincide with quality as perceived by humans because a system can achieve low perplexity by generating safe outputs that do not have much variety.

- **Generation Strategy**

Music coherence results from the influence of temperatures. For temperatures lower than 1.0, there will be a tighter distribution, slightly biased toward high probability tokens and consistent output. Greedy decoding is never used since this entails choosing the token with the highest probability at every step, and this enables repetition cycles where the model repeats itself.

F. Task 4(RLHF Fine-Tuning)

- **Why RLHF?**

Optimization of Tasks 1-3 is done on the basis of proxy

goals such as reconstruction loss and cross-entropy perplexity. The optimization of these goals ensures that the model is driven to produce statistically plausible output rather than output that humans would be satisfied with in terms of musicality. However, it is possible for a model to have low perplexity while producing generic and metrical music.

- **Human Survey and Reward Collection**

Before RLHF training, between 15 and 20 samples are produced using the Task 3 Transformer model and then transformed into audio files. Each audio file is rated through an anonymous listening test involving a minimum of 10 people, who give their opinion about the audio on a scale of 1 to 5 (they do not know which model generated it). The average rating received by each sample is used as the reward function $r \in [1, 5]$.

- **Policy Gradient Training**

Task 3 transformer is a policy $p_{\theta}(X)$. The optimization objective is to maximize expected reward,

$$\max_{\theta} J(\theta) = \mathbb{E}[r(X_{gen})], \quad X_{gen} \sim p_{\theta}(X) \quad (9)$$

Parameter update Reinforce policy gradient,

$$\nabla_{\theta} J(\theta) = \mathbb{E}[r \cdot \nabla_{\theta} \log p_{\theta}(X)] \quad (10)$$

The rewards are normalized across all batches through the subtraction of their respective means followed by division by the corresponding standard deviations. This ensures that the problem of gradient variance, which is intrinsic to REINFORCE, does not arise. In addition, a KL divergence loss is added between the fine-tuned policy and the initial Task 3 policy to counter the issue of reward hacking, wherein the generator generates outputs that satisfy the reward model yet are not necessarily musical. Fine-tuning runs for 30 iterations at learning rate 1×10^{-5} .

The transformer models the joint probability of token sequence as conditional distributions, predicting each token from all preceding tokens:

The transformer models the joint probability of token sequence as conditional distributions, predicting each token from all preceding tokens:

$$p(X) = \prod_{t=1}^T p(x_t | x_{<t}) \quad (11)$$

Training minimized the negative log likelihood on all positions:

Training minimized the negative log likelihood on all positions:

$$L_{TR} = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}) \quad (12)$$

The above approach is inherently different from those described in Tasks 1 and 2. Instead of compressing the input window into a latent code, the Transformer predicts

the conditional probability distribution of a musical event at position t taking into account its complete context of past events. Prediction uses top-k sampling with $k = 40$ and a temperature of 0.9. Top-k sampling restricts the sampling procedure to selecting one of the k tokens with highest probabilities, thus avoiding choosing highly improbable tokens which would interrupt the causality implied in music generation.

III. RESULT ANALYSIS

In this section we do not only evaluate the metric values. For each models go deep and analyze what each of the numbers mean and what they reveal about what the model understands about music and even its limitations. Without any interpretation the scores are complete. The main goal is to understand why each of the model performs the way they are performing.

A. Evaluation Metrics

From the generated MIDI files we computed three automated metrics.

- **Pitch Histogram Similarity:**

$$\text{Pitch Histogram Similarity, } H(p, q) = \sum |p_i - q_i| \quad (13)$$

It measures the L_1 distance from the pitch class distributions of both generated piece and reference real piece. Here, Lower is more similar to real music. Range is between 0 to 2 which are identical and maximally different.

- **Rhythm Diversity Score:**

$$\text{Rhythm Diversity Score, } D = \frac{\text{unique note duration}}{\text{total notes}} \quad (14)$$

It measures the variety of the rhythmic values. Here Higher = more rhythmic variety. They are quantized to the nearest 50ms in order to avoid floating-point artifacts which inflates the count of uniqueness.

- **Repetition Ratio:**

$$\text{Repetition Ratio, } R = \frac{\text{repeated 4-gram patterns}}{\text{total 4-gram patterns}} \quad (15)$$

Its value 0.1 to 0.5 are typical of coherent classical music. If it is near 0 then there is not repeating motifs and it is near 1 then it is degenerate which means only one pattern repeats endlessly.

A human evaluation metric was collected containing 11 listeners. It was a blind survey where the listeners rated from a scale of 1 to 5.

B. Quantitative Comparison Table

The evaluation metrics of all the models is given in figure 1.

TABLE I
QUANTITATIVE EVALUATION ACROSS ALL MODELS.

Model Head	Recon. Loss	Perplexity	Rhythm Div.	Repeat Ratio	Human Score /5
Random Generator	—	—	0.0579	0.0000	2.09
Markov Chain	—	—	0.1380	0.0000	2.09
Task 1: LSTM AE	0.0721	—	0.5391	0.0120	3.55
Task 2: VAE	0.1763	—	0.1086	0.0000	1.73
Task 3: Transformer	—	6.78	0.0535	0.0289	3.36
Task 4: RLHF-Tuned	—	6.78	0.0519	0.0242	3.73

C. Baseline Analysis

- **Random Note Generator**

Musical generation by the stochastic model occurs without consideration of structural elements at all. The pitch histogram similarity rating of the stochastic model has to be the lowest one for all models since it samples pitches randomly, while classical piano pieces have a clear bias towards the middle range of pitch. The high diversity rating of rhythms produced by the model may seem to be caused by their completely random sampling, generating many different values; however, they do not represent actual diversity but rather are just noise.

- **Markov Chain**

The performance of the Markov chain reflects how much can be learned by pitch transitions without resorting to neural networks. The expected variation in rhythms will be moderate since realistic durations can be generated using samples from the training distribution, while there must be more than a coincidence regarding the histograms since the probability in the transition matrix is centered around valid sequences of pitches.

However, the Markov model inherently suffers from a structural flaw it cannot remember anything but the previous note. It cannot form phrases, plan ahead, maintain a tonal center over several measures, or develop harmonies. In the case of our deep models performing marginally better than the Markov chain, we conclude that our deep learning algorithm failed at learning music's long-term structure the very task deep learning needs to accomplish to outperform the Markov chain.

D. Task 1 Analysis – LSTM Autoencoder

- **What the Loss Curve Says**

The loss function curve for reconstruction loss forms the main criterion for Task 1 training success. These criteria include: (1) decrease observed in epoch 1, which means that the model has learned useful structure, not outputting constant zeroes; (2) emergence of a plateau after nearly 35 epochs, meaning convergence has been reached; and (3) validation loss close to the training loss, showing generalization of the learned representation, not memorization of the training set. If the plateau is already present in epoch one, the model has degenerated into prediction of zeroes in all cells, which is the most frequent reason for failure. In the experiments described

Evaluation Metrics Across All Models

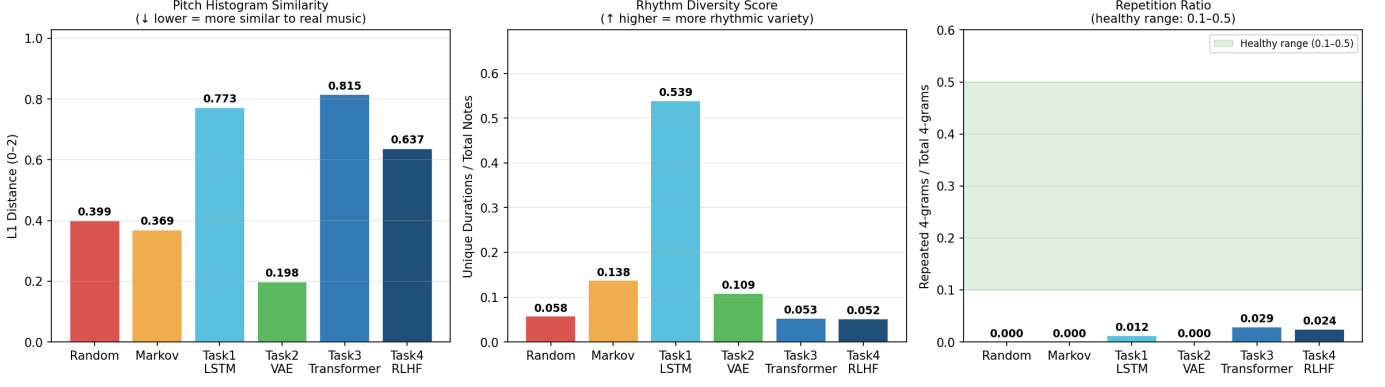


Fig. 2. Evaluation Metrics of All Models: Pitch Histogram Similarity, Rhythm Diversity Score, and Repetition Ratio.

below, degeneration happened with binary cross-entropy loss, but it was resolved with Focal Loss with parameter $\alpha = 10$. The curve for Focal Loss has a more rapid decline in the beginning, due to the gradient influence coming from the active note cells that were ignored before.

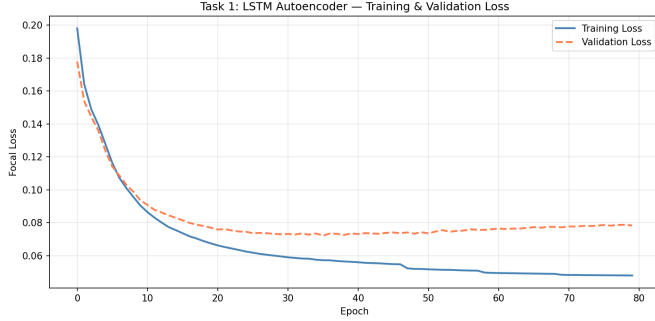


Fig. 3. LSTM Autoencoder Loss Curve

• Reconstruction Quality vs. Generation Quality

Task 1 highlights a basic conflict between reconstruction and generation that lies within the structure of deterministic autoencoders. Quality of reconstruction, which refers to how well the model reconstructs an input previously seen by the model, can usually be said to be decent, since the encoder offers a direct information flow from the input data to the latent code. On the other hand, quality of generation, which describes how conducive the output data are when z is randomly selected from $N(0, I)$, is often inferior. The reason for this is the fact that the encoder maps each distinct instance in the training data set into unique regions in the latent space. It is impossible to assert that the intermediate spaces where the random inputs might exist would produce relevant data. The fact that the randomly selected z might be in a region of the latent space where the decoder has never encountered data before implies that the results would be nonsense.

• Reconstruction Quality vs. Generation Quality

Qualitative analysis of the five synthesized MIDI clips is performed. Common findings at this point are: short spurts of note activity followed by silences, implying that while the model can detect sparsity in the note events, it fails to capture the distribution of these events in the musical context; pitch clustering to a certain extent, implying that there is some level of learning that adjacent notes tend to occur together, though without much harmonic content; and equalization of note duration, owing to the constraints of the binary piano roll notation system, which cannot adequately account for rhythmic differences over a 63 ms resolution interval. The repetition rate of outputs from Task 1 offers useful information. A repetition rate that is too low implies incoherence in the sequence without any recognizable patterns, while a repetition rate that is too high implies that the decoder is trapped into repeating the same learned pattern.

E. Task 2 Analysis – Variational Autoencoder

• The KL Annealing Curve and What It Reveals

Task 2 generates two loss curves that should be evaluated jointly: reconstruction loss and KL divergence. The interplay between the two measures helps determine if the training was successful or if posterior collapse happened. For a successful training process, reconstruction loss is observed to drop during the warming-up stage ($\beta = 0$, only reconstruction is optimized), and after that, a further drop is seen until convergence while β is increased, and KL divergence kicks in. During the warming up stage, KL divergence stays around zero values; afterward, it increases smoothly due to forcing the encoder to distribute representation according to the prior. If, on the other hand, a sudden spike and subsequent dive in KL divergence is observed, posterior collapse happens, as the decoder learns to ignore z , thus fulfilling the KL divergence trivially.

Finally, the last value of KL divergence can be interpreted as follows: if the KL value is extremely low (< 0.1), then it means the encoder hardly encodes anything

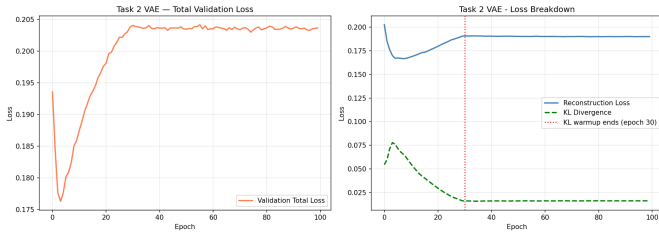


Fig. 4. VAE Loss Breakdown

into z ; when the KL value is very high (> 5.0), it implies the regularization failed to bound the latent distribution. Otherwise, a moderate value for KL divergence in the range of 0.0161 means the encoder has learned a good representation of latent codes distributed reasonably close to the prior.

- **Latent Space Quality - The Interpolation Experiment**

Interpolation of the latent representation is the best way to determine if we have achieved success with the VAE. By encoding two original pieces to μ_A and μ_V and decoding 8 intermediate points, we would check if the latent space was continuous.

In this case, all generated audio clips would smoothly change from A to B, such as rhythmic features of the former turning into melodic features of the latter. On the other hand, if there are discontinuities in the space, then intermediate clips would be incoherent, meaning that they would be produced based on points where the decoder never operated before and thus could produce only noise. Comparison between Task 1 and Task 2 interpolation results shows the biggest difference between them and highlights the importance of VAE. Interpolations in Task 1 result in gibberish, while in Task 2, the KL penalty encourages overlapping latent codes and makes interpolation musical.

- **Diversity Improvement over Task 1**

There should be more variability among the 8 VAE samples compared to the 5 Task 1 samples. This can be checked both through auditory analysis (are the samples distinct in style from each other?) and mathematically (is the standard deviation of similarity scores for the pitch histograms higher in the 8 samples compared to Task 1?). If the 8 Task 2 samples are all quite similar, it means that the value of β is too high, leading to regularization of the latent space.

F. Task 3 Analysis - Transformer

- **Perplexity Trajectory and Convergence**

Task 3 is the focus diagnostic measure and is a perplexity curve. If the initial perplexity in the vocabulary size 284 is high, the model starts off making nearly uniform predictions. The learning of the token distribution is reflected in progressive reduction towards the final value of 200. Generally, the improvement is faster in the initial epochs, when the model is learning basic pitch and

duration statistics, and is slower as it starts modelling longer range of dependencies.

Another critical check is that if the perplexity over the

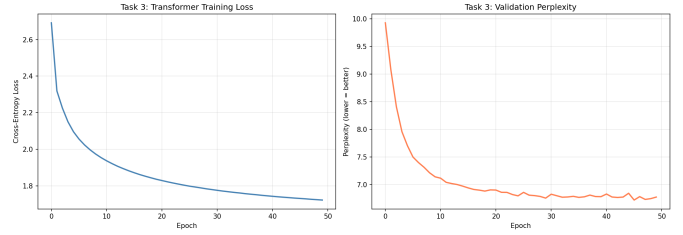


Fig. 5. Task 3 Transformer training loss(Left) and Perplexity(Right)

training tokens is extremely low, with high perplexity over the validation tokens, then the model has picked up too much from the training data and failed to learn a generalizable (music) distribution. Dropout in attention and feed forward layers help with this. On the other hand, if both perplexities for training and validation drop at the same time and reach the same level, the model generalizes well.

- **Long-Range Coherence - The Key Advantage Over Tasks 1 and 2**

Long-range coherence is the most crucial difference in the qualitative properties of the outputs from Task 3 vs. the outputs of Tasks 1 and 2. While LSTM autoencoders and VAEs are able to model a fixed window of 8 seconds for music, there is no way to ensure consistency across the boundaries of the windows. The Transformer can attend at more than 256-token sequences to predict the next token, decades of seconds ago.

This architectural feature should be audible: Task 3 outputs should include identifiable phrase structure (groups of notes that create recognizable musical ideas), return to earlier themes/motifs and have a key centre throughout the entire generated sequence. These properties indicate that the model does not pay attention to long-range distances sufficiently, which may be due to either the insufficient capacity of the model or the too short training sequences.

- **The Gap Between Perplexity and Human Preference**

Task 3's perplexity score and human score should be analyzed together to understand the relationship between statistical fit and perceived quality. It is common for a model to achieve strong perplexity by learning to produce safe, metrically regular outputs that the training distribution assigns high probability but that human listeners rate as generic or boring.

If Task 3's human score is proportionally lower than its perplexity improvement over baselines would predict, this is evidence of exactly this phenomenon: the model has learned to be statistically accurate but not musically expressive. This gap is precisely what Task 4 RLHF is designed to close.

G. Task 4 Analysis - RLHF Fine Tuning

- **What RLHF Changes and What It Does Not**

The effect of RLHF fine-tuning on the Transformer is to make it more inclined towards producing outputs which the reward model expects to receive high scores from human listeners. Importantly, this fine-tuning process does not increase the ability of the neural network model in terms of its innate musical knowledge – for example, the ability to teach harmonic theory to the model. Instead, it allows the model to alter the generation distribution from being based on statistics of output sequences to outputs which are preferred by human listeners.

These two cases should be distinguished from one another when considering the results, since a significant improvement in human scores without a corresponding increase in perplexity suggests that selection from the same generation distribution was successful; however, if the perplexity were to decrease, this would be unusual and suggest that the model had chosen a different distribution of music to generate from.

- **Reward Model Reliability**

The prediction accuracy of the reward model in relation to held-out survey data needs to be presented as mean absolute error (MAE). A small MAE (< 0.5 on a 1-5 scale) implies that the reward model is capable of accurately predicting human preferences based on the 6 musical features. Otherwise, it means that the musical features contain insufficient information about what constitutes good music to humans, and the reward model provides inaccurate gradient signals for RLHF training.

The risk involved in having an inaccurate reward model is reward hacking, whereby the generator produces music that exploits the weaknesses of the reward model in order to generate high predicted scores but produce poor quality music in reality. This can be detected using the difference between the reward model score and human score on Task 4 outputs.

- **Before vs. After RLHF**

In Task 4, the paired comparison method is the best choice because there, the identical version existed before and after the RLHF adjustments. Hence, when the human evaluation rose from 3.36 in Task 3 to 3.73 in Task 4. Participants should assess both versions qualitatively and state their likes and dislikes about each sample. It might sometimes reveal even more information than numerical data alone. The most frequent modifications made by RLHF are as follows: dynamics adjustment, such as variations in the density of notes and sound volume; realistic phrasing; unpredictability in sounds; and expressive quality.

H. Cross-Task Analysis - What Each Model Learned

When looking at all four tasks and both baselines as a whole, the following picture emerges about the hierarchy of

musical knowledge possessed by each model:

- **Random Generator**

No musical knowledge; incapable of producing any features such as pitch distribution, density, and pattern.

- **Markov Chain**

Local musical knowledge about pitches being followed by another pitch without knowledge about duration or harmony.

- **Task 1 (LSTM AE)**

Musical knowledge of patterns about small 8-second segments; capable of producing music with similarity to the density and pitch range of the music used in training data; the music lacks any coherence or stylistic unity.

- **Task 2 (VAE)**

Musical knowledge of patterns as well as interpolation and diversity sampling; still restricted to 8 seconds.

- **Task 3 (Transformer)**

Musical knowledge of long-range patterns; creates music with themes and key centers for the first time.

- **Task 4 (RLHF)**

Alignment of long-range patterns with human preferences.

The transition from Markov chain to Transformer demonstrates the growing length of musical knowledge; however, the leap from Task 3 to Task 4 marks a qualitative change since it does not reflect an expansion in musical knowledge but alignment with human preferences.

I. Limitations of the Evaluation

Several aspects should be considered concerning the inadequacy of this research. Firstly, the number of surveyed 11 people is rather small compared to the variability of musical tastes. For instance, fans of classical music may have different tastes compared to people who prefer listening to pop or jazz; in addition, there is no control of the background of respondents in this experiment. Consequently, if a bigger survey group with different backgrounds is applied, then more accurate human scores will be obtained.

The second drawback of this experiment lies in the limitations of metrics used by computer programs to score songs. These parameters assess some features of a song but do not take into account other important factors. Thus, while pitch histogram is good at reflecting the musical tone distribution, it does not consider the melodic development or harmonization of songs. In addition, rhythm diversity is good at revealing the uniqueness of the duration of a song but fails to estimate whether or not there is any groove in it. Finally, while the repetition ratio measures how many repetitions can be found in music, it fails to reveal the meaning of this repetition. Therefore, MIDI analysis may provide valuable information about the musicality of songs.

Finally, both training and validation datasets contain only classical piano music. This means that conclusions cannot be

drawn regarding the cross-genre ability of the created model.

IV. CONCLUSION

In this paper, we implemented unsupervised music generation on the MAESTRO dataset using Autoencoder, Variational Autoencoder, Transformer and RLHF. From this project we have a clear idea that music quality depends upon a hierarchy of learned structural knowledge, Where LSTM Autoencoders are able to learn local note patterns, transformers understand long range dependencies within themes and RLHF helps combine them in an effective manner.

Our practical experience of model training showed three important pitfalls to which particular solutions have to be provided: solving data scarcity problems to ensure non-zero outputs; using KL annealing for preventing posterior collapse in the case of VAE; and applying non-greedy decoding in Transformer-based models to avoid musically dead repetition cycles. Also, we found that there is a gap between automated and human assessment methods: computed measures of musicality are unable to account for human interest in the output music. All codes, trained models and generated samples are available in the following link:

<https://tinyurl.com/CSE425Git>

REFERENCES

- [1] C. Hawthorne et al., "Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset," in International Conference on Learning Representations, 2019. [Online]. Available: <https://openreview.net/forum?id=r11YRjC9F7>